

Actor-Critics

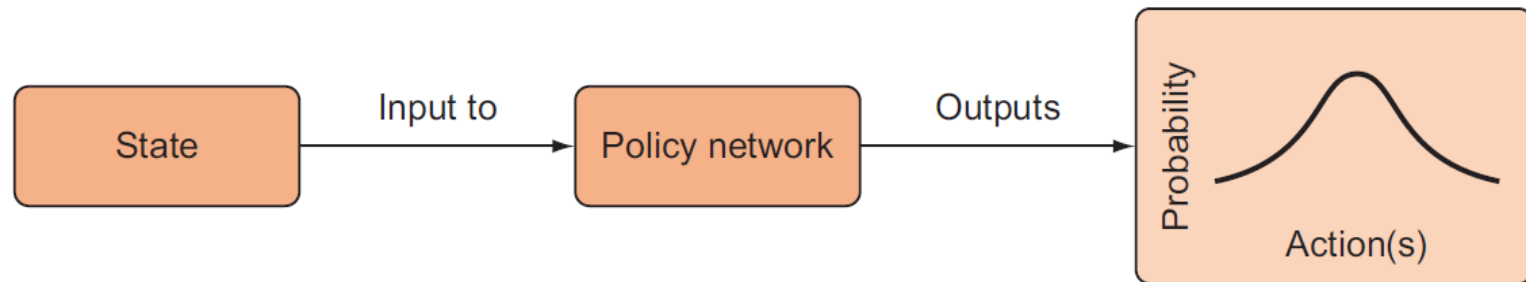
Dr. Mongkol Ekpanyapong

Actor-Critics

- AC
- A3C
- A2C
- DDPG

Policy Gradient Method

- We introduce Policy Gradient Method that take state as input and return probability distribution over actions



Policy Gradient

Advantages:

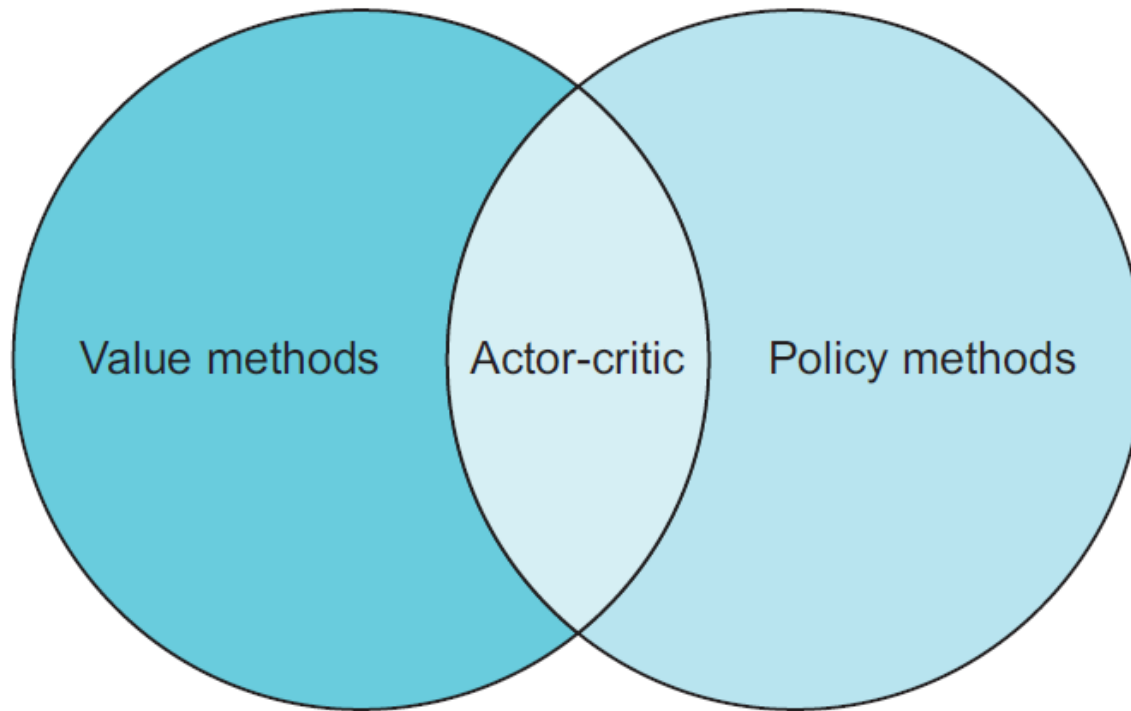
- Predict policy directly
- Support continuous actions

Disadvantages:

- Rely on Monte Carlo Method
- Unstable

Actor-Critic methods

- Combined approach

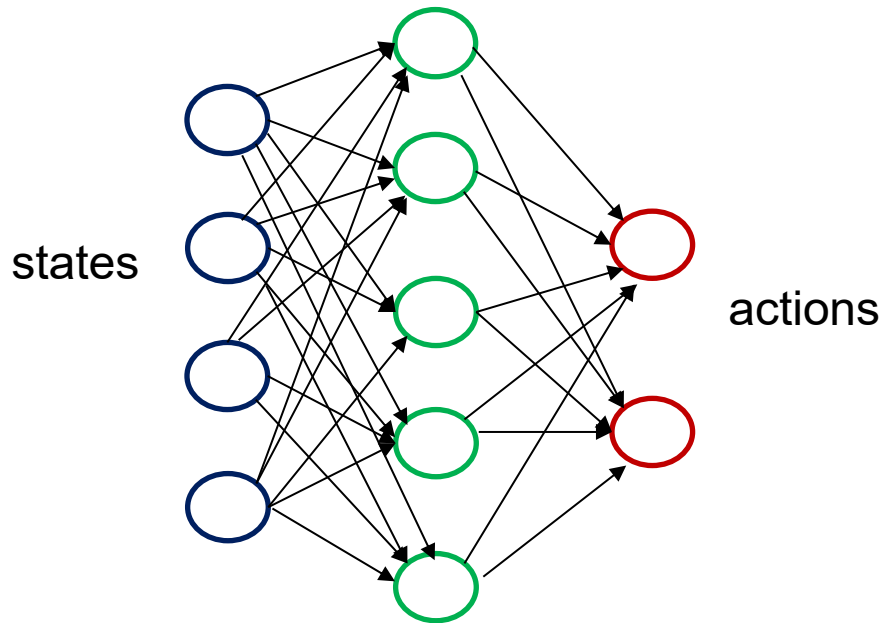


Grid world example

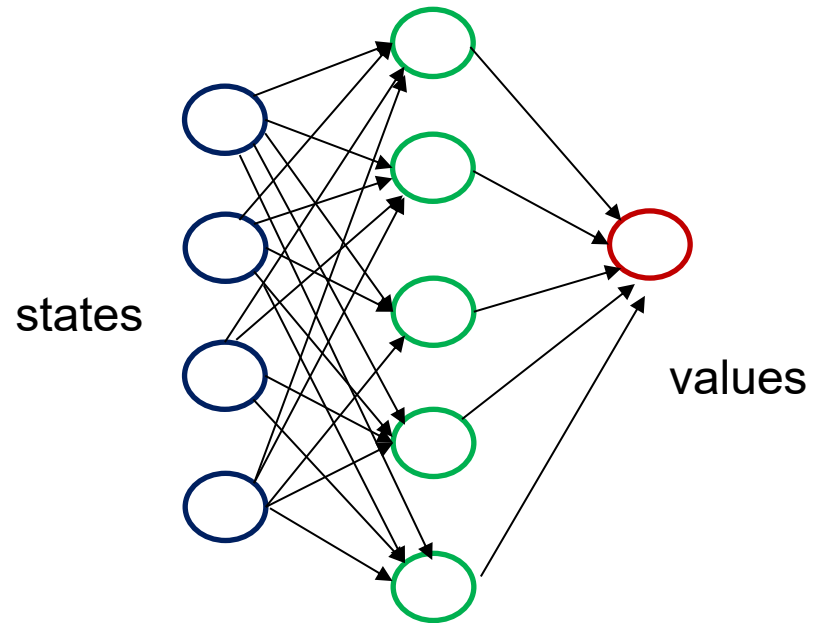
- In Grid world, the reward is -1 on every move except for the end of episode.
- This is hard for policy gradient to learn
- Then, we use value lookup to get the value function

Actor-Critics

Actor (Policy Gradient)



Critics (Value learning)



Paper: Addressing function approximation errors in actor-critics method[2018]



From Policy Gradient Theorem

$$\nabla_{\theta} J(\theta) = \mathbb{E} [G_t \nabla_{\theta} \ln \pi_{\theta}(a_t|s_t)]$$

$$\theta = \theta + \nabla_{\theta} J(\theta)$$

$$\theta = \theta + \alpha \gamma^t G_t \nabla_{\theta} \ln \pi_{\theta}(a_t|s_t)$$

Policy Gradient Method

Maximize:

$$\nabla_{\theta} J(\theta) = E [G_t \nabla_{\theta} \ln \pi_{\theta\pi}(a_t|s_t)]$$

Modify to:

$$\nabla_{\theta} J(\theta) = E [Q_{\pi}(s_t, a_t) \nabla_{\theta\pi} \ln \pi_{\theta\pi}(a_t|s_t)]$$

$$\theta_{\pi} = \theta_{\pi} + \nabla_{\theta\pi} J(\theta)$$

$$\theta_{\pi} = \theta_{\pi} + \alpha Q_{\pi}(s_t, a_t) \nabla_{\theta\pi} \ln \pi_{\theta\pi}(a_t|s_t)$$

Value function

$$L_v = (G - V(s, \theta_v))^2$$

- If G is total return, then it could be Monte Carlo algorithm
- If G comes from bootstrapping, then it is based on TD

Note that:

$$Q_\pi(s_t, a_t) \approx r_{t+1} + \gamma V_{\theta_v}(s_{t+1})$$



Pseudo Codes

```
state = env.get_state()
policy = actor(state)
action = policy.sample()
next_state, reward, done = env.step(action)
q_value = critic(state, action)
next_policy = actor(next_state)
next_action = next_policy.sample()
next_q_value = critic(next_state, next_action)
actor_loss = -1 * policy.logprob(action) * (q_value.detach())
target = reward + gamma * next_q_value * (1 - done)
critic_loss = pow(q_value - target.detach(), 2)
loss = actor_loss + critic_weight * critic_loss
loss.backward()
```



A3C

- Asynchronous Advantage Actor-Critic
- Proposed by Google DeepMind team
- We've studied Policy Gradient
- We've studied Q-Learning
- Actor: Policy
- Critic: Value function

From Policy Gradient Theorem

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\Sigma_{t=0}^{\infty} \gamma^t Q(s_t, a_t) \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t)]$$

$$\theta = \theta + \nabla_{\theta} J(\theta)$$

$$\theta = \theta + \alpha \gamma^t G_t \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t)$$

Policy Gradient Network

- Optimal action

$$\pi(a \mid s, \theta_p) = \text{NeuralNet}(\text{input} : s, \text{weights} : \theta_p)$$

- Policy loss

$$\nabla_{\theta_p} J(\theta) = \mathbb{E} [\sum_{t=0}^{\infty} \gamma^t A(s_t, a_t) \nabla_{\theta_p} \ln \pi_{\theta_p}(a_t | s_t)]$$

- Value loss

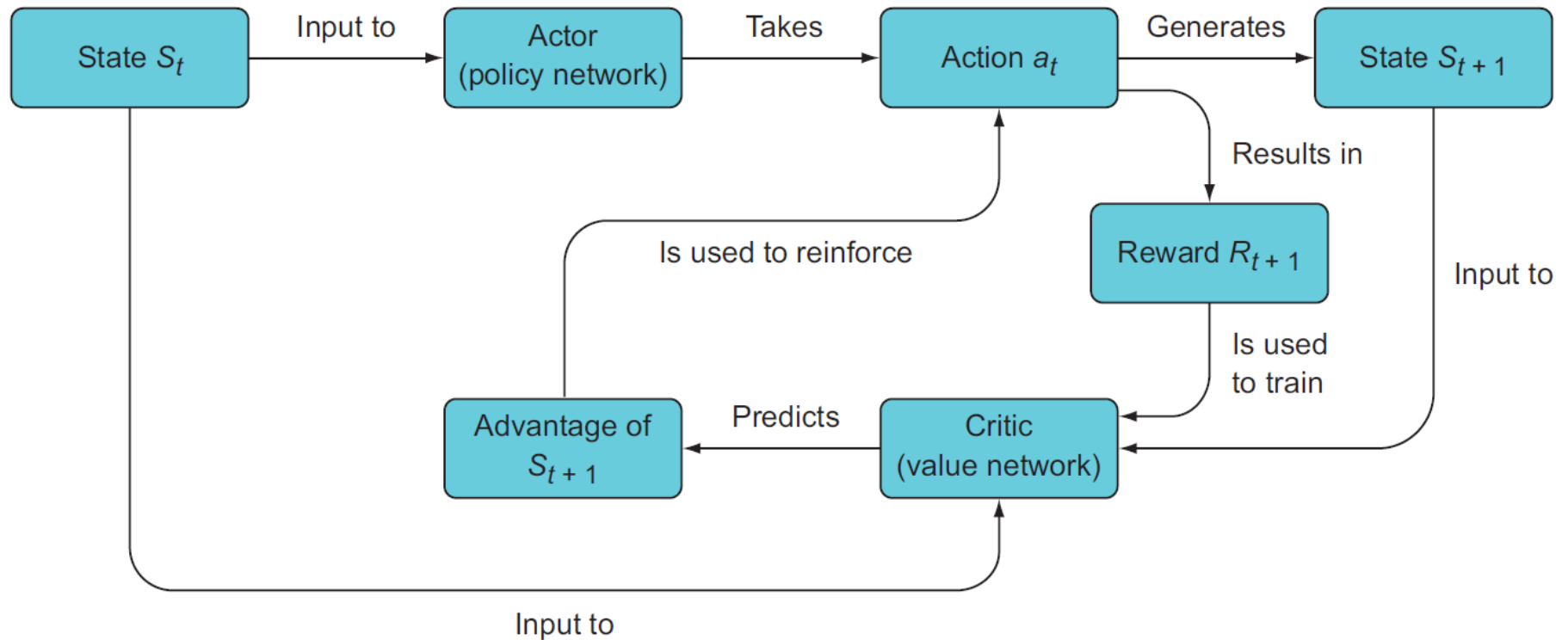
$$L_v = (G - V(s, \theta_v))^2$$

Advantage Function

$$\nabla_{\theta p} J(\theta) = \mathbb{E} [\Sigma_{t=0}^{\infty} A(s_t, a_t) \nabla_{\theta p} \ln \pi_{\theta p}(a_t | s_t)]$$

$$A(s, a) = Q(s, a) - V(s) \quad \text{or} \quad A(s, a) = R + \gamma V(s') - V(s)$$

Actor-Critic Model



Improvement

- We want to improve the sample efficiency by updating more frequently
- We want to decrease the variance of the reward we used to update our model

Parallel learning

Environment #1

Environment #2

Environment #3

Environment #4

...

Environment #n

$s_{1,t}$

$s_{2,t}$

$s_{3,t}$

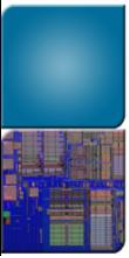
$s_{4,t}$

$s_{n,t}$

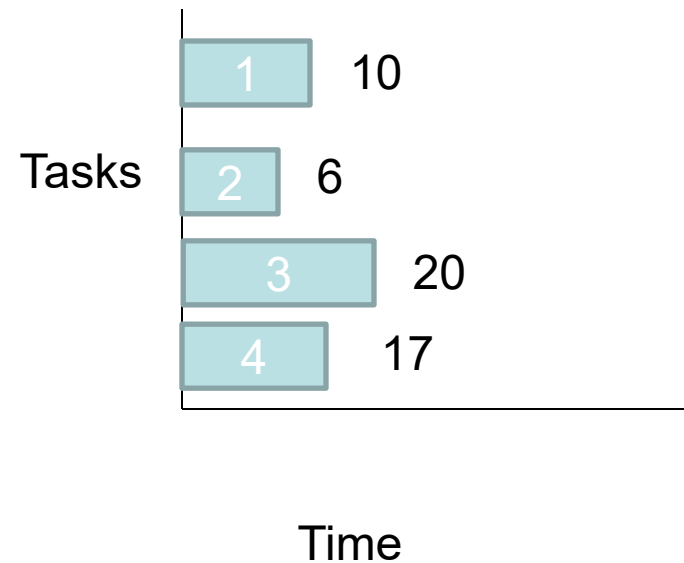
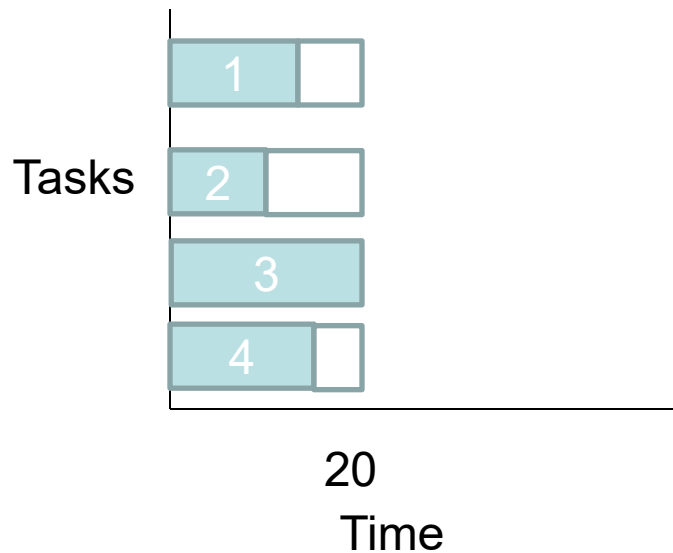
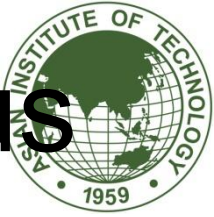
$S_t =$

$$\begin{bmatrix} s_{1,t} \\ s_{2,t} \\ s_{3,t} \\ s_{4,t} \\ \vdots \\ s_{n,t} \end{bmatrix}$$

Agent

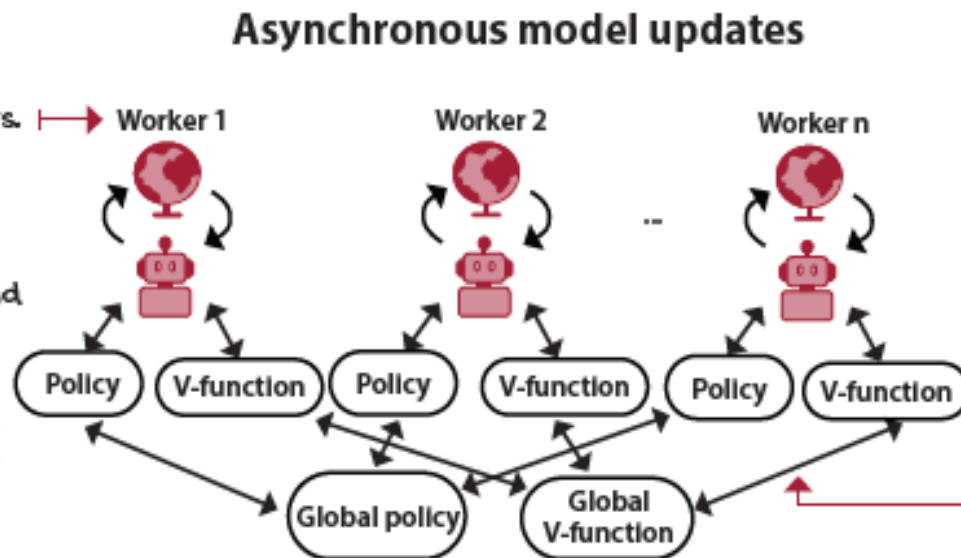


Synchronous vs. Asynchronous Computing



Asynchronous Computing

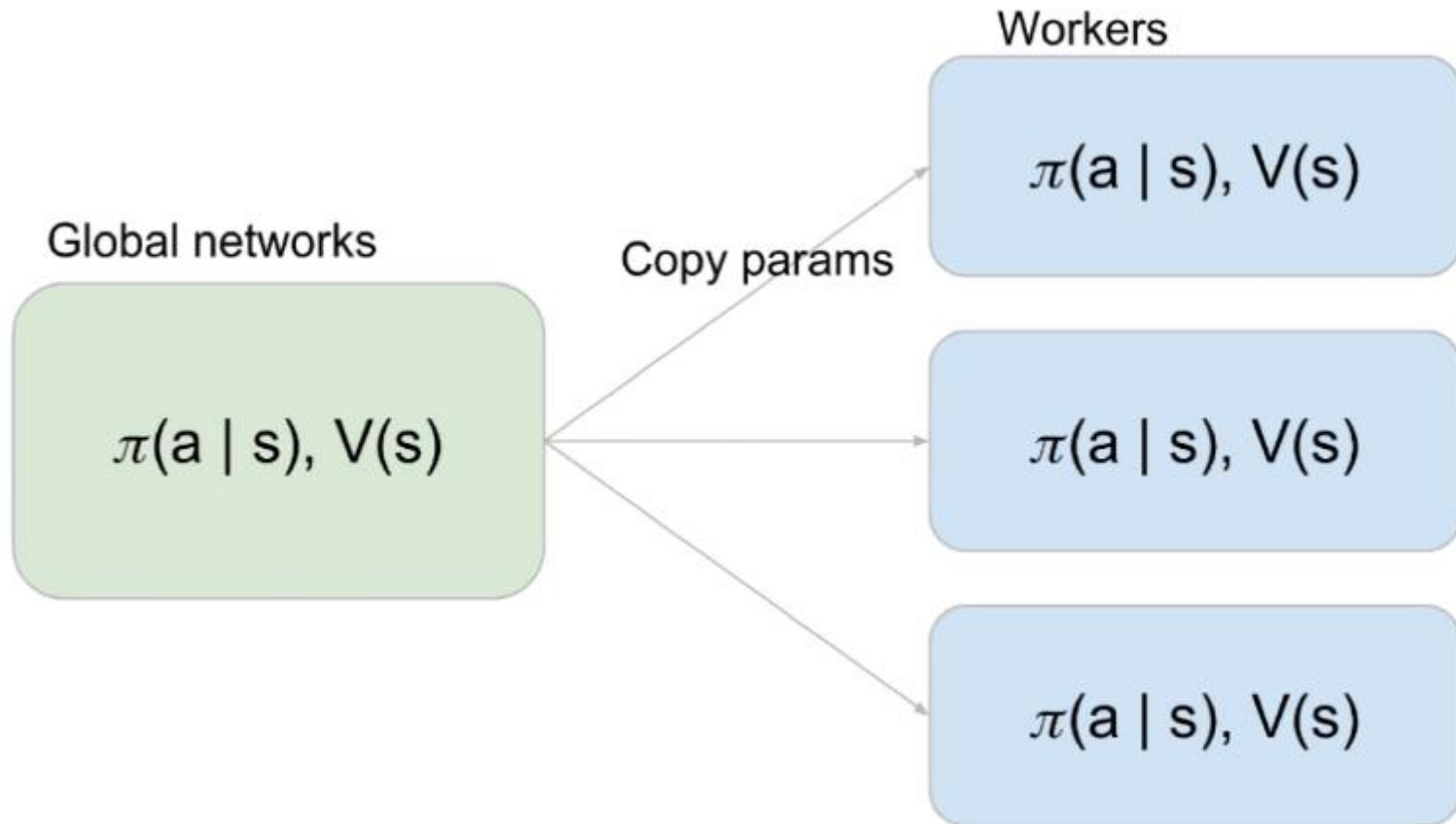
(i) In A3C, we create multiple worker-learners. Each of them creates an instance of the environment and the policy and V-function neural network weights use for generating experiences.



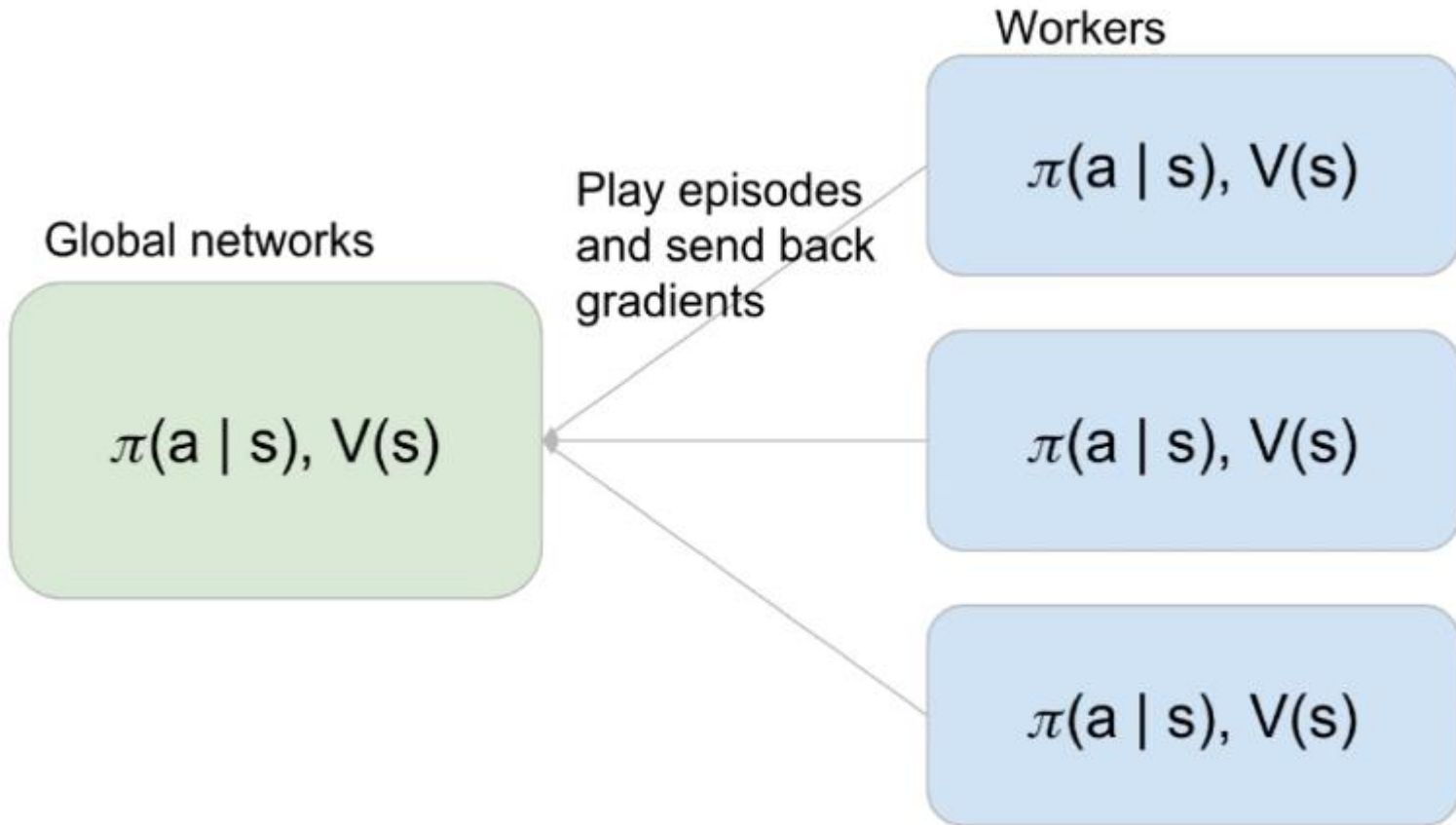
(a) After an experience batch is collected, each worker updates the global model asynchronously, without coordination with other workers. Then, they reload their copy of the models and keep at it.



A3C



A3C





A3C

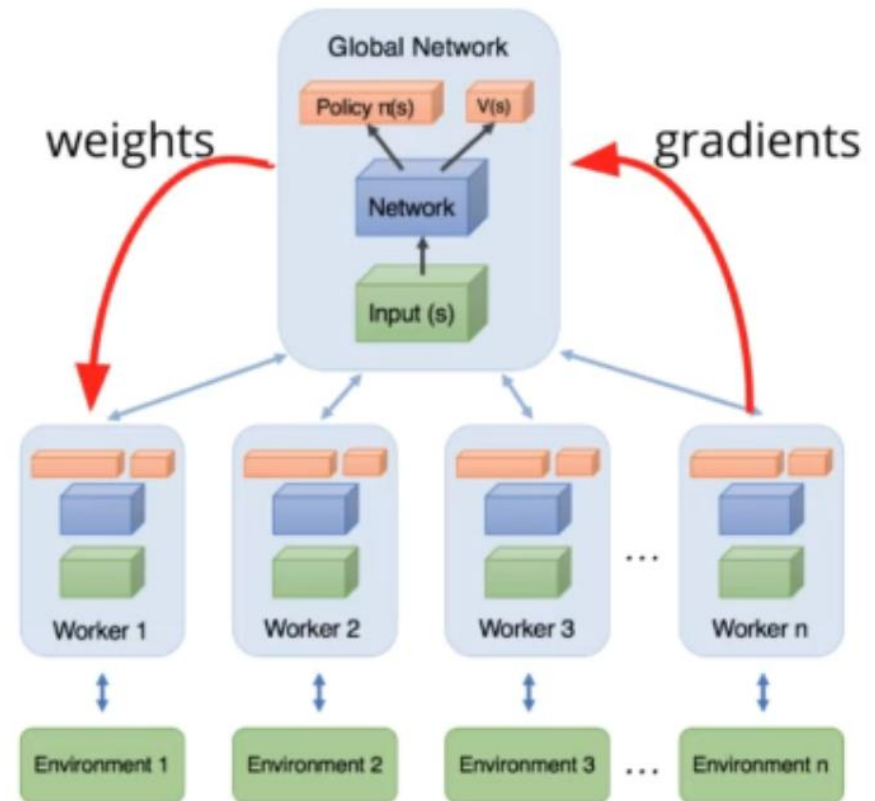
- The workers do all the work and combine to the global network
- Workers play episodes, calculate errors and update the gradient back to the global network
- The update could be in parallel, speeding up the work



A3C Strategy

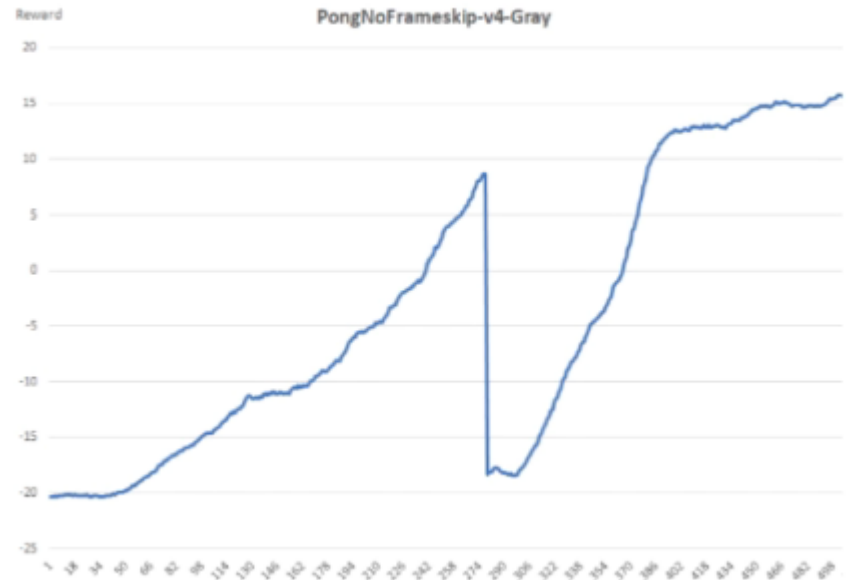
- Master copies its weights to worker
- Workers do the learning
- Every 5 steps, each worker sends back the gradient

Problem: some out-of-date weight



Stability

- Each episode will be random
- Each action is sampled stochastically and independent
- We can get performance drop because of the bad update



N-Step return

- The algorithm use N-Step return
- $N = 3$

$$V(s) = r + \gamma r' + \gamma^2 r'' + \gamma^3 V(s''')$$

What is Entropy?

- Entropy is defined as a measure of randomness or disorder of a system

$$H(p) = - \sum_i p_i \ln p_i$$

Example: Entropy

Consider a coin toss (head : x_0 and tail : x_1)

$$p(X=x_0) = 1 \quad \text{and} \quad p(X=x_1) = 0$$

$$H(x) = -[1 \ln(1) + 0 \ln(0)] = 0$$

Example: Entropy

Consider a coin toss (head : x_0 and tail : x_1)

$$p(X=x_0) = 0 \quad \text{and} \quad p(X=x_1) = 1$$

$$H(x) = -[0 \ln(0) + 1 \ln(1)] = 0$$

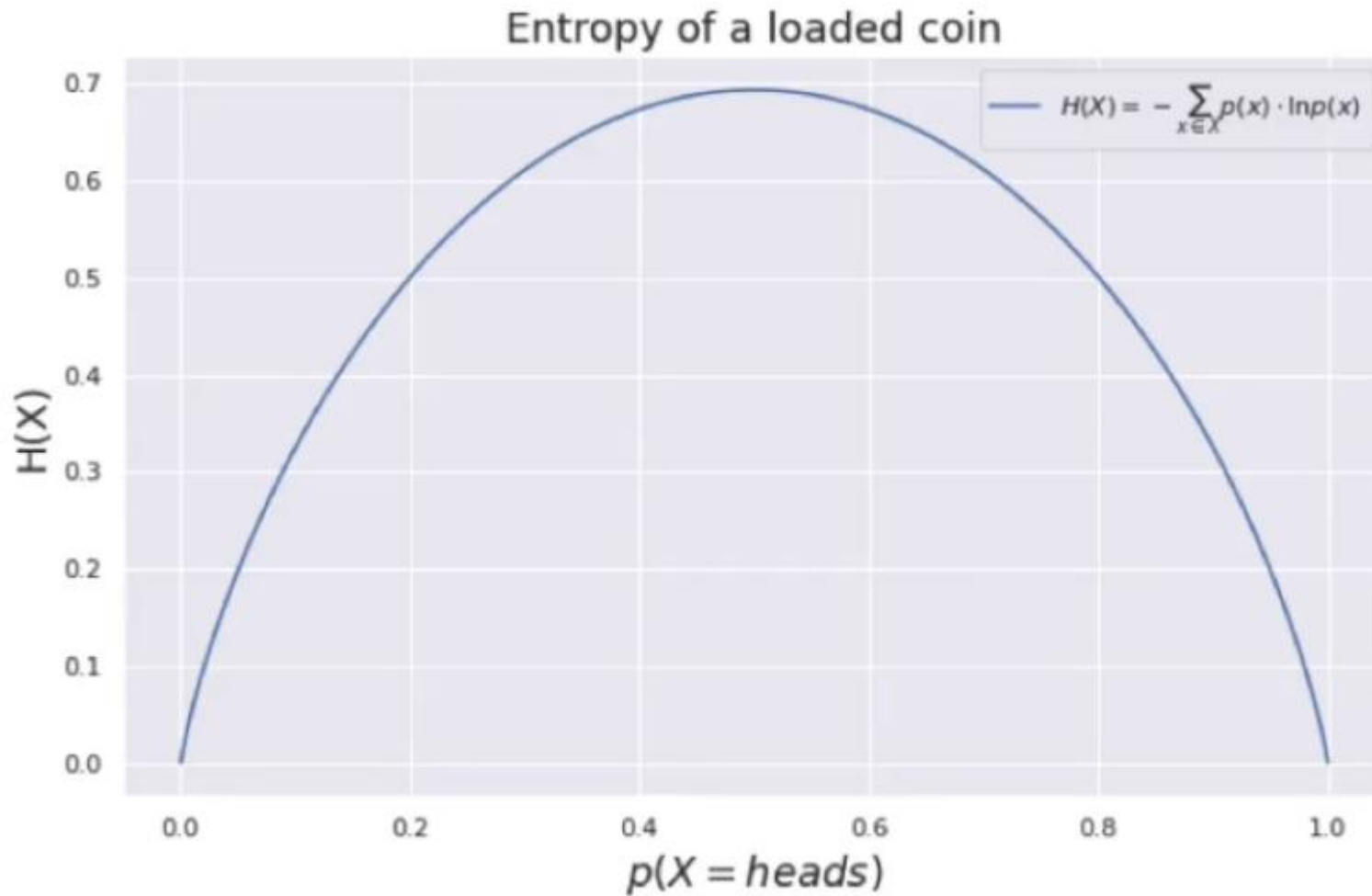
Example: Entropy

Consider a coin toss (head : x_0 and tail : x_1)

$$p(X=x_0) = 0.5 \quad \text{and} \quad p(X=x_1) = 0.5$$

$$H(x) = -[0.5 \ln(0.5) + 0.5 \ln(0.5)] = -0.69$$

Entropy



Two decorative squares in the top-left corner: a solid blue square and a purple square with a circuit-like pattern.

Entropy in our policy

- To encourage exploration, entropy is introduced in the loss function
- Entropy in our policy

$$H(\pi) = - \sum_i \pi_i \ln \pi_i$$



From Policy Gradient Theorem

$$\nabla_{\theta} J(\theta) = E [\Sigma_{\tau=t+1}^T Q(s_t, a_t) \nabla \ln \pi_{\theta}(a_t | s_t)]$$

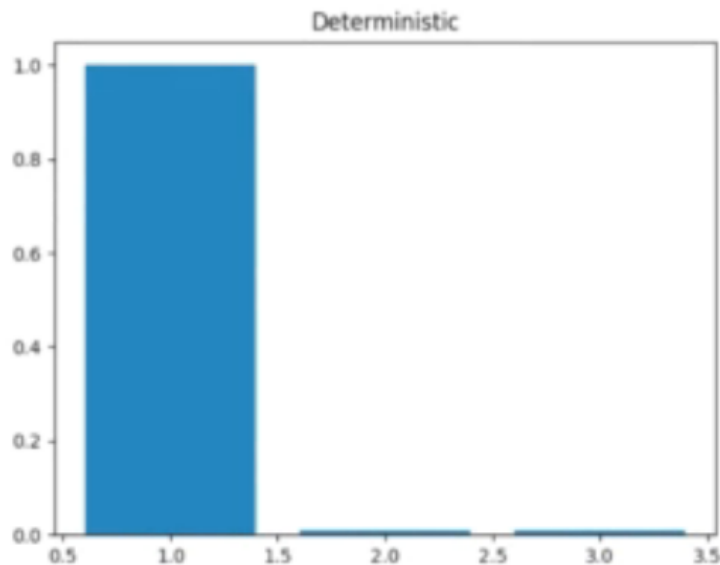
$$\theta = \theta + \nabla_{\theta} J(\theta)$$

$$\theta = \theta + \alpha \gamma^t \nabla_{\theta} A(s, a) \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) + \beta \nabla H(\pi)$$

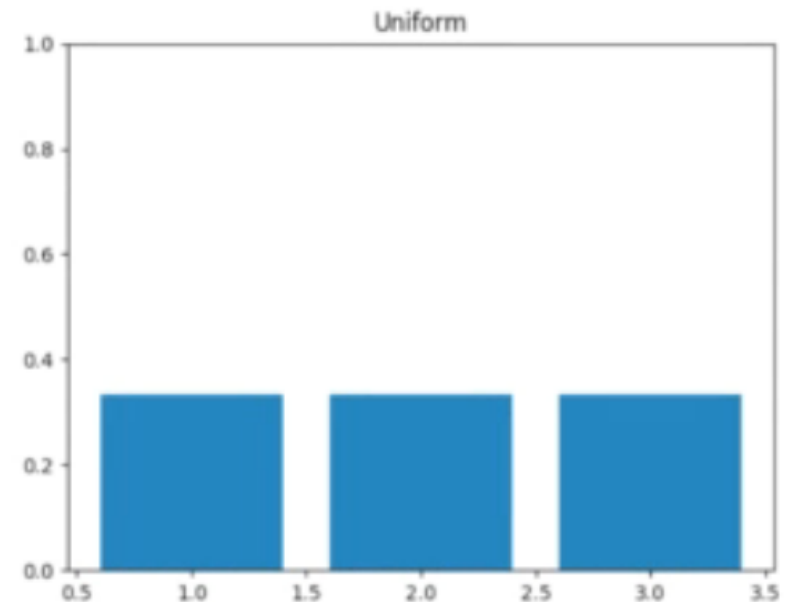


Exploration vs. Exploitation

No exploration



Maximum exploration



A decorative graphic in the top left corner consisting of a blue square above a grid of smaller squares in various colors (blue, green, yellow, red).

A3C Summary

- Advantage function
- Asynchronous Computing
- Entropy regularization
- N-Step return



A2C

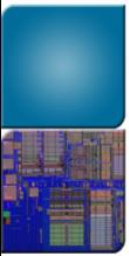
- Proposed by the OpenAI company in a blog
- Synchronous computing
- No need to share weight
- No Asynchronous update problem
- N-Step return = 5

In the top left corner, there are two decorative squares. The top one is a solid blue square, and the bottom one is a purple square with a circuit-like pattern.

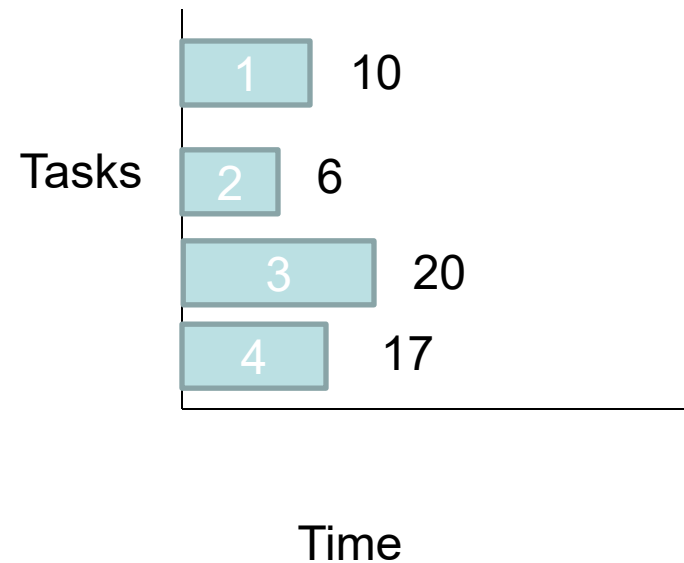
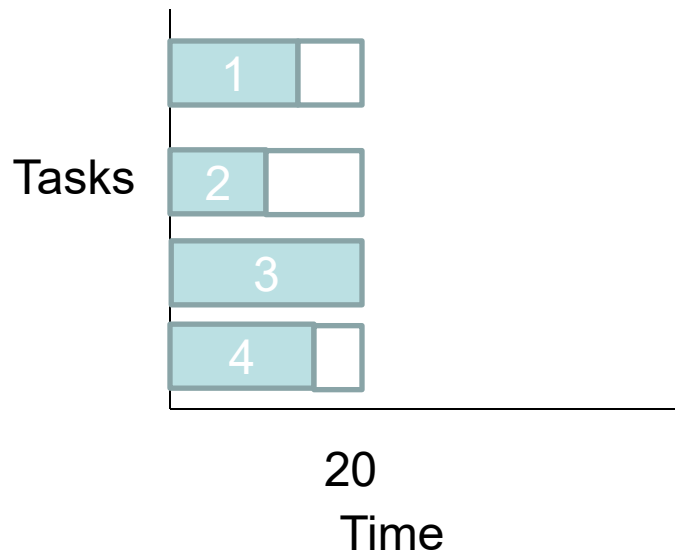
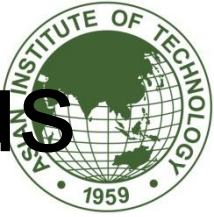
A2C Strategy

- We run 5 steps in parallel on each worker
- Each work returns 5 of (s, a, r, s')
- This data is used to update the master network
- Only one copy of the network is needed





Synchronous vs. Asynchronous Computing



Policy Gradient Method

$$\nabla_{\theta} J(\theta) = E [G_{\pi}(s) \ln \pi_{\theta}(a_t|s_t)]$$

Modify to in AC, A2C, and A3C:

$$\nabla_{\theta} J(\theta) = E [V_{\pi}(s) \ln \pi_{\theta}(a_t|s_t)]$$

$$\nabla_{\theta} J(\theta) = E [Q_{\pi}(s) \ln \pi_{\theta}(a_t|s_t)]$$

$$\nabla_{\theta} J(\theta) = E [A_{\pi}(s) \ln \pi_{\theta}(a_t|s_t)]$$

DDPG

From paper “Continuous control
with deep reinforcement learning”

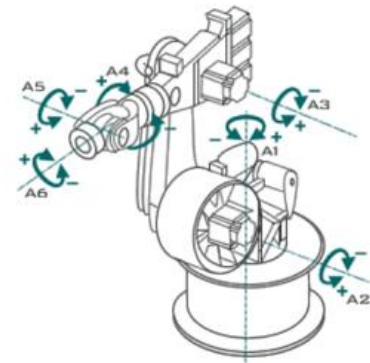
DDPG

- DPG was proposed in 2014
- It is designed to improve over DQN to address continuous action space
- It is also supported deterministic action so that it could be used for control applications

Drawback of DQN

- We could use binning to handle continuous action space
- If we have let's say only output 3 action per each degree of freedom $[-1, 0, 1]$, the action of neural network between 0.8 and 0.9 will get the same treatment
- By changing action value from one part, usually it will impact other part. But, by using binning, we have less choice to control
- If we have 6 degree of freedom and 10 action per each degree of freedom, we create 60 possible actions
- Performance of ANN is relatively depending on number of output

Image from robotpark.com





How can we modify DQN?



Remember this in DeepQ:

State-action-in-value-out architecture

State variables in

- Cart position
- Cart velocity
- Pole angle
- Pole velocity at tip

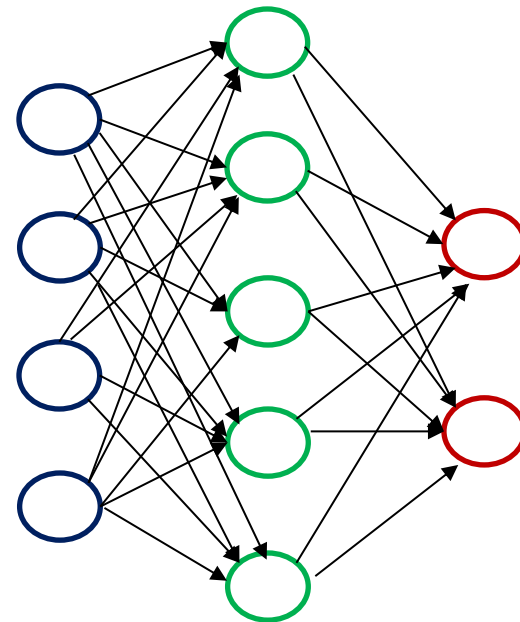
State s . For example,
[-0.1, 1.1, 2.3, 1.1]

Action in

action a , for example, 0.

This could also be represented as a
one-hot vector, for example, [1, 0].

value out
 $Q(s,a)$, for
example, 1.44



Deterministic Policy

- We define:

$$\mu(s) = \arg \max_a Q(s,a)$$

- Hence, we assume that $\mu(s)$ is the result of maximize function. It is the best current greedy action.



Deep Deterministic Policy Gradient (DDPG)



- Given the state, we select the continuous and differentiable policy function (deterministic policy):

$$\mu(s)$$

- We find the parameters of the policy that results in highest Q value when θ is the weight in policy network

$$\theta_{\mu} = \theta_{\mu} + \alpha \nabla_{\mu} Q(s, \mu(s))$$



Deterministic Policy

- For Reinforce, when you get the output of the policy

$$\pi(a=0|s) = 0.8$$

Then, it is 80% chance picking this action

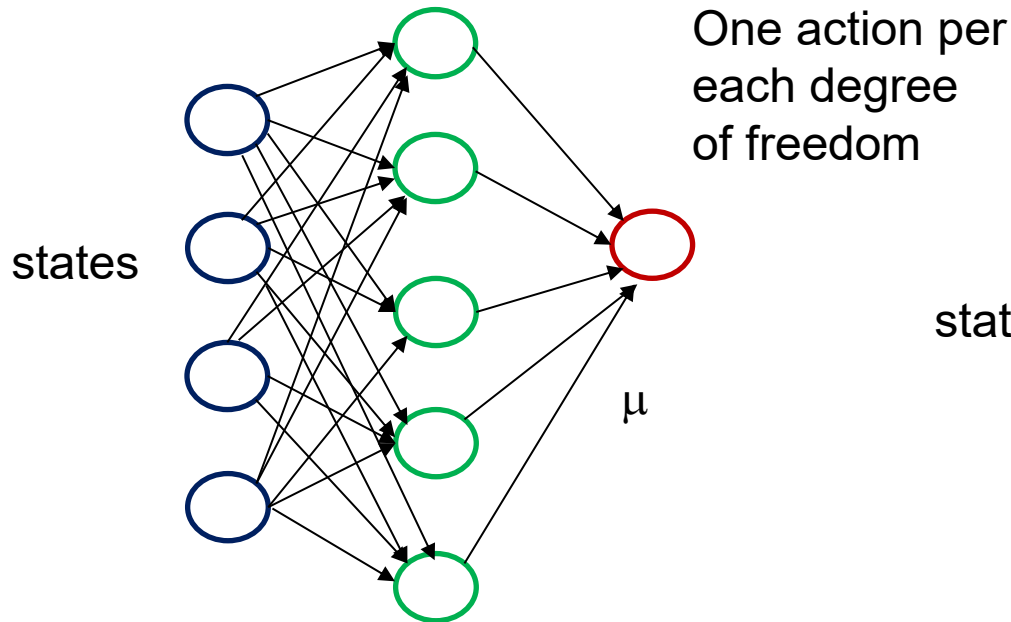
- For DDPG

$$\mu(s) = 0.8$$

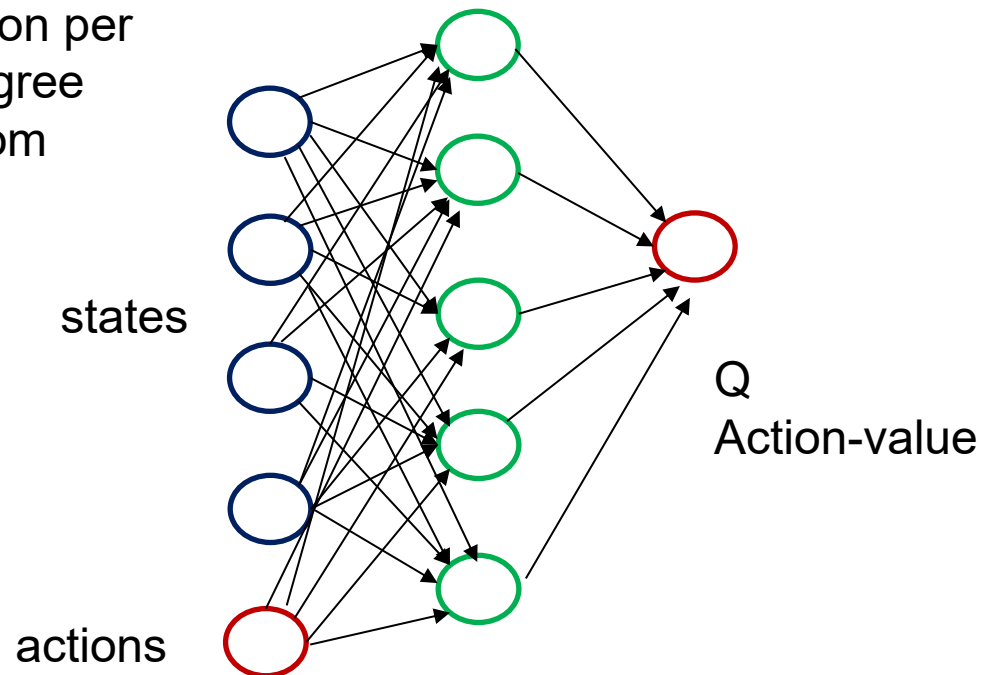
Then, it means set the value to 0.8 e.g., turning steering wheel by 0.8 radians

DDPG

Actor (Policy Gradient)



Critics (Value learning)



Paper: Continuous control with deep reinforcement learning





Deep Q-Learning Loss Function

$$L(\theta) = \frac{\left[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{Target}} - \underbrace{Q(S_t, A_t)}_{\text{Prediction}} \right]^2$$



DDPG Critic Loss Function

$$L(\theta_Q) = E [r + \gamma Q'(s_{t+1}, \mu'(s_{t+1}, \theta_{\mu'})) - Q(s_t, a_t)]^2$$

μ is the output of policy function, θ_{μ} is the policy parameter and ' is for the target (red)



Actor: Learning the policy

$$J(\theta_{\mu}) = E [Q(s_t, \mu(s_t, \theta_{\mu}))]$$



A decorative image in the top-left corner consisting of a blue square above a circuit board pattern.

Deep Q Network Features

- Uses target network
To update the main network or target network for both policy network and critic network
- Uses a replay buffer



Basic algorithm

$s = \text{env.reset}()$

while not done:

$a = \mu(s)$

$s', r, \text{done} = \text{env.step}(a)$

update θ_Q

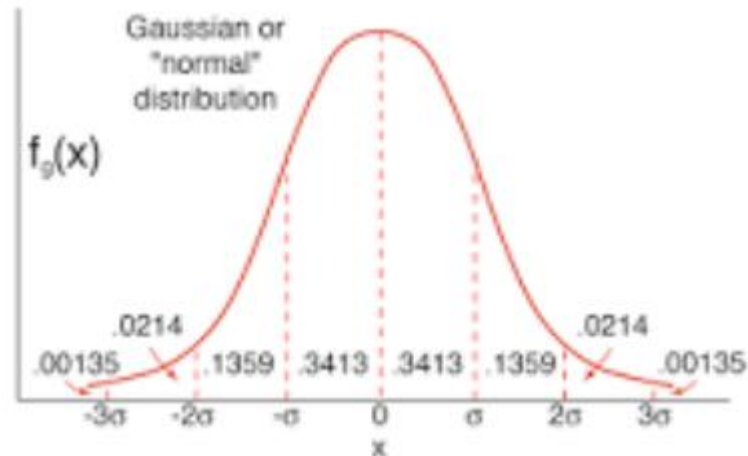
update θ_μ

$s = s'$

Ignore Experience replay buffer and target network (just for now).

How to perform exploration?

- Policy μ is deterministic
- μ usually means the mean of something
- Add gaussian noise in the policy function



Target Network

We will have target network for both

- Policy network
- Critic network

Target Network Update

$$\theta^{\mu}_{targ} \leftarrow \tau \theta^{\mu}_{targ} + (1 - \tau) \theta^{\mu}$$

$$\theta^{\mathcal{Q}}_{targ} \leftarrow \tau \theta^{\mathcal{Q}}_{targ} + (1 - \tau) \theta^{\mathcal{Q}}$$



Pseudo code

Initialize θ_μ , θ_Q , replay_mem, θ_μ^{arg} , θ_Q^{arg}

$s = \text{env.reset}()$

while not done:

$a = \mu(s) + \text{noise}$

$s', r, \text{done} = \text{env.step}(a)$

$\text{replay_mem.add}(s, a, r, s', \text{done})$

 input, target = sample from replay_mem

 update θ_Q using $(\text{targ} - \text{pred})^2$

 update θ_μ using $Q(s, \mu(s))$

 update target network for both policy and critic

$s = s'$



Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for t = 1, T **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

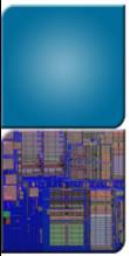
Question?

- How many set of Neural Network we have for DDPG?

LunarLander

- Target is the origin (0,0)
- Game ends if the lander crashes or come to the rest
- Action: Do nothing, Fire the left engine, Fire the main engine, Fire the right engine, Fire main engine
- Reward: 200 point (landing), -100 crash, Fire main engine -0.3, Each leg contact is 10





Questions?

